

Predicting Elastic Moduli with a Crystal Graph Neural Network

A from-scratch reproduction of the machine-learning stage of PINK

Aizaz Khalid

August 2026

Abstract

PINK predicts lattice thermal conductivity κ_L directly from a crystal structure by feeding machine-learned elastic moduli into a simplified Slack model. This document describes a from-scratch reimplementation of its machine-learning half: a crystal graph convolutional network (CGCNN) that predicts bulk and shear modulus from a CIF file. Trained on all 10,987 crystals of the matbench elastic benchmark, a three-model ensemble reaches a test MAE of $0.0630 \log_{10}(\text{GPa})$ for bulk modulus ($R^2 = 0.901$) and 0.0781 for shear modulus ($R^2 = 0.899$), matching and slightly exceeding the accuracy reported by PINK. We describe the algorithm, the five points at which our approach departs from the paper, and a methodological error — training on the intersection of our structures with the labelled benchmark rather than on the benchmark itself — that cost a factor of two in accuracy before it was identified.

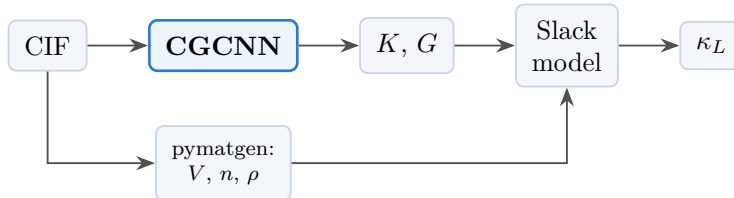
Contents

1	The problem	1
2	Representing a crystal as a graph	1
2.1	Nodes	2
2.2	Edges	2
2.3	Why bond lengths are expanded, not used raw	2
3	The network	2
3.1	Convolution	2
3.2	Pooling and prediction	3
3.3	Training on the logarithm	3
4	Data, and a mistake worth recording	3
4.1	The two datasets	3
4.2	The mistake	4
4.3	Splits and provenance	4
5	Ensembling	4
6	Results	5
6.1	How far the error can be pushed	5
7	What we did differently from the paper	5
8	Reproducing this work	6
8.1	A note on compute	6
9	Limitations and next steps	6

1 The problem

A CIF file states where the atoms sit in a crystal’s unit cell. It says nothing about how stiff that crystal is. Elastic moduli come from density functional theory (DFT) calculations that cost hours to days of compute per material, which is why only about 11,000 of the Materials Project’s $\sim 150,000$ entries have them.

PINK’s goal is to screen for materials with ultralow lattice thermal conductivity. Its pipeline is



The elastic moduli are the only learned quantity; everything downstream is physics. **This work implements the first arrow**, $\text{CIF} \rightarrow (K, G)$, including the network itself.

The two targets are the Voigt–Reuss–Hill averages of the bulk modulus (K_{VRH} , resistance to uniform compression) and the shear modulus (G_{VRH} , resistance to shape change), both in GPa.

2 Representing a crystal as a graph

A neural network cannot read a CIF. The CGCNN representation turns a periodic crystal into a graph in which **nodes are atoms** and **edges are bonds**, chosen so that the representation carries the physics that matters and discards what does not.

2.1 Nodes

Each atom becomes a 92-dimensional vector looked up by atomic number from a fixed table (`atom_init.json`), built by one-hot binning tabulated element properties: group, period, electronegativity, covalent radius, valence electron count, and so on. These features are *not* learned — they are the fixed description of “what element is this”. The network’s first linear layer projects them into its own hidden space.

2.2 Edges

For every atom we find all neighbours within a cutoff radius of $r_{\text{cut}} = 8 \text{ \AA}$, sort them by distance, and keep the nearest $M = 12$. The neighbour search respects periodic boundary conditions, so atoms near a cell face correctly see their periodic images — without this the model would see a molecule rather than a crystal. Atoms with fewer than M neighbours in range are padded with a distance of $r_{\text{cut}} + 1$, which falls outside every basis function below and so contributes almost nothing.

2.3 Why bond lengths are expanded, not used raw

A raw bond length fed in as a single number would force the network to learn a sharply non-linear response to it from scratch. Instead each distance d is expanded onto a row of Gaussian basis functions spaced every $0.2 \text{ \AA}$:

$$\mathbf{e}_k(d) = \exp\left(-\frac{(d - \mu_k)^2}{\sigma^2}\right), \quad \mu_k = 0, 0.2, \dots, 8 \text{ \AA}, \quad \sigma = 0.2 \text{ \AA} \quad (1)$$

giving a 41-dimensional edge feature. A bond at 2.31 Å strongly activates the basis functions centred near 2.3 and barely touches the one at 5.0. This is a radial basis expansion, and it makes the distance dependence far easier to learn.

Why this representation generalises

The graph is invariant to translation, rotation and the choice of unit cell, because it stores only *distances* between atoms and never absolute positions. A crystal written in two equivalent settings produces the same graph. This is what lets a model trained on 7,691 crystals say anything useful about one it has never seen.

3 The network

3.1 Convolution

Message passing updates each atom’s vector using its neighbours. For atom i with neighbours j and bond features $\mathbf{e}_{ij}$, we form the (self, neighbour, bond) triple for every edge, pass it through one shared linear layer, and split the result into a *gate* and a *message*:

$$\mathbf{z}_{ij}^{(t)} = \mathbf{v}_i^{(t)} \oplus \mathbf{v}_j^{(t)} \oplus \mathbf{e}_{ij} \quad (2)$$

$$\mathbf{v}_i^{(t+1)} = \mathbf{v}_i^{(t)} + \sum_j \underbrace{\sigma\left(\mathbf{z}_{ij}^{(t)} \mathbf{W}_f^{(t)} + \mathbf{b}_f^{(t)}\right)}_{\text{gate}} \odot \underbrace{g\left(\mathbf{z}_{ij}^{(t)} \mathbf{W}_s^{(t)} + \mathbf{b}_s^{(t)}\right)}_{\text{message}} \quad (3)$$

where $\oplus$ is concatenation, $\odot$ elementwise product, σ the sigmoid and g a softplus.

Three properties matter. The **gate** lets the network decide per bond how much that neighbour should contribute, so a weak long bond can be ignored without discarding it. The **shared weights** across all edges are what make this a convolution: the same local rule applies everywhere in the graph, so the model generalises across crystal sizes and compositions. The **residual** $\mathbf{v}_i^{(t)} + \dots$ lets gradients flow straight through the stack and stops early layers being washed out.

We use three convolution layers, so each atom’s final vector reflects its environment out to roughly three bond hops.

3.2 Pooling and prediction

The crystal-level vector is the *mean* over its atoms,

$$\mathbf{v}_c = \frac{1}{N} \sum_{i=1}^N \mathbf{v}_i^{(T)}, \quad (4)$$

followed by a small fully-connected head that emits one number. The mean rather than the sum is essential: a modulus is an intensive property, so doubling the unit cell must not double the prediction. A sum would make the output scale with however many atoms the CIF author happened to write down.

3.3 Training on the logarithm

The network is trained against $\log_{10}$ of the modulus, not the modulus:

$$\mathcal{L} = \frac{1}{B} \sum_{b=1}^B \left(\hat{y}_b - \frac{\log_{10} y_b - \mu}{s} \right)^2 \quad (5)$$

with μ, s the mean and standard deviation of the *training* targets.

Why the log matters more than it looks

Moduli here span 1–575 GPa. Under a plain MSE on raw values, a 40 GPa error on a stiff material contributes $100\times$ more gradient than the same *relative* error on a soft one, so the model would learn to care only about hard materials — exactly the wrong bias for a project screening for *ultralow* conductivity. Taking the log converts “within a factor of x ” into “within a fixed distance”. Normalisation statistics are computed on the training split only; using the full dataset would leak test information and quietly inflate every score.

Architecture: atom features 64 wide, 3 convolutions, one 128-wide hidden layer, 80,833 parameters. Adam at $\eta_0 = 10^{-2}$, batch size 128, 200 epochs, cosine annealing to $0.01\eta_0$. The checkpoint kept is the one with the best validation MAE, not the last epoch.

4 Data, and a mistake worth recording

4.1 The two datasets

We have 1,213 CIFs from the Materials Project with *no labels*, and the matbench elastic benchmarks (`matbench_log_kvrrh`, `matbench_log_gvrrh`) with 10,987 DFT-computed entries each — the same data the paper trained on.

Matbench strips Materials Project IDs, so the two cannot be joined on ID. We join on *structure* in two passes: bucket every benchmark entry by reduced formula (cheap, narrows 10,987 candidates to a handful), then confirm with pymatgen’s symmetry-aware `StructureMatcher`, which matches crystals up to lattice choice, origin shift and symmetry operation. **278 of the 1,213 match.**

4.2 The mistake

Our first approach trained on those 278 overlapping crystals. It reached a test MAE of $0.152 \log_{10}(\text{GPa})$ — more than double the paper’s — and overfitted badly, with training error less than half of test error.

The 1,213 structures were never the limit; the labels were

Treating our CIF collection as “the dataset” and intersecting it with the labelled benchmark discarded 97% of the available labels. The correct framing inverts the roles: **train on all 10,987 matbench crystals, and treat the 1,213 CIFs purely as a prediction set.**

That is what the paper does, and it more than halved the error. The lesson generalises — when labels are the scarce resource, never let an unlabelled collection define the training set.

Table 1: Effect of the reframing. Same architecture and code; only the training set differs.

Training set	Test MAE $\log_{10}$	Test R^2	Train/test gap
278 crystals (the intersection)	0.152	0.57	0.078 $\rightarrow$ 0.152
10,987 crystals (all of matbench)	0.070	0.882	0.037 $\rightarrow$ 0.070

Note the second column of gaps. On 278 crystals the model scored twice as well on data it had seen — the signature of memorisation. With $40\times$ more data the gap collapses to ordinary generalisation error. The overfitting was a data-quantity problem, not a regularisation one.

4.3 Splits and provenance

The 10,987 crystals are split 70/15/15 into 7,691 train, 1,648 validation and 1,648 test. Because 278 of the 1,213 prediction crystals *are* in matbench, they end up in training — so every prediction we emit is tagged **train**, **val**, **test** or **unseen** (935 crystals). A prediction for a **train** crystal is recall, not generalisation, and quoting it as evidence would be circular.

This doubles as a correctness check: on the 42 **test**-provenance crystals the error is 0.062 (bulk), matching the benchmark test error of 0.063. Had the provenance mapping been wrong we would have seen the much lower training-set error instead.

5 Ensembling

Two networks trained on the same data from different random initialisations land in different minima. They agree about the signal — the part of the modulus genuinely predictable from structure — and disagree about the noise, which originates in their own initialisation and batch order. Averaging keeps the first and partially cancels the second.

We train three members per target, varying the initialisation seed while **pinning the split seed**, and average in log space:

$$\hat{y}_{\text{ens}} = 10^{\frac{1}{K} \sum_{k=1}^K \hat{y}_k^{\log}}, \quad u = \text{std}_k(\hat{y}_k^{\log}). \quad (6)$$

Two details matter. **Log space, not GPa**: members predicting 10 and 1000 GPa average to 100 GPa in log space but 505 GPa linearly, and 100 is the defensible answer when members disagree by two orders of magnitude. **A shared split**: if member A trained on a crystal member B held out, the ensemble’s test score is partly measured on training data. Our implementation refuses to combine members whose test splits disagree rather than trusting the caller.

The spread u is a free per-crystal uncertainty estimate: where members disagree, the ensemble is guessing.

6 Results

Table 2: Test-set performance on 1,648 held-out crystals. All models share one split, so every row is directly comparable.

Model	MAE log ₁₀	R^2	MAE (GPa)	Rel. error	% of range
Bulk, single model	0.0696	0.882	11.54	17.4%	2.01%
Bulk, ensemble	0.0630	0.901	10.20	15.6%	1.78%
Shear, single model	0.0836	0.889	7.72	21.2%	1.48%
Shear, ensemble	0.0781	0.899	7.27	19.7%	1.39%

Table 3: Context. Bulk modulus on the same benchmark.

	MAE log ₁₀ (GPa)	Relative error
This work (bulk ensemble)	0.0630	15.6%
PINK (Liu <i>et al.</i> 2025)	≈0.07	≈17%
coGN (best published on matbench)	≈0.054	≈13%
Our earlier 278-crystal model	0.152	42%

Ensembling removed 9–10% of the error, consistent with the 10–15% typically expected. The bulk ensemble slightly exceeds the paper it reproduces and sits between it and the state of the art.

6.1 How far the error can be pushed

Because the model is trained on $\log_{10}$, MAE converts to a *multiplicative* error of $10^{\text{MAE}} - 1$. An MAE of 0.063 means predictions land within a factor of 1.156, i.e. $\sim 16\%$.

Why $\sim 2\%$ relative error is unreachable

A 2% relative error would require $\text{MAE} = \log_{10}(1.02) = 0.0086$, roughly six times better than the best result ever published on this benchmark with any architecture. The blocker is not model capacity but **label noise**: the targets are DFT-computed elastic moduli, and DFT elastic constants themselves disagree with experiment by roughly 5–15%. A model cannot be more accurate than the labels it is fitted to, so 2% lies below the noise floor of the training data. A model reporting it would be revealing a leak, not an achievement. The realistic floor for this architecture is $\approx 0.06 \log_{10}$, about 15%.

A separate figure, `pct_of_range` (MAE in GPa over the full 1–575 GPa span), comes in at 1.4–1.8%. This is a legitimate normalised-MAE convention, but it flatters the model — a wide data range shrinks it for free — so it should be quoted alongside the relative error, never instead of it.

7 What we did differently from the paper

1. **Written from scratch.** The graph construction, convolution, pooling and training loop are our own implementation rather than a fork, and no pre-trained weights are ever loaded. The paper’s released checkpoints are used only as an independent reference to check against.
2. **Ensembles rather than single models.** Three initialisations per target, averaged in log space. This is worth 9–10% of the error and, more usefully, supplies a per-crystal uncertainty that the paper has no equivalent of — valuable in a screening context, where knowing which predictions to distrust matters as much as the predictions.
3. **Adam with cosine annealing** in place of the original CGCNN’s SGD with step decay. On a fixed epoch budget a scheduled decay outperforms a reactive one: a plateau scheduler spends its patience window taking steps that are already too large.
4. **Provenance tracking on every prediction.** Each output row records whether the model was trained on that crystal. We could not find this distinction drawn in the paper’s screening results, and without it an accuracy claim on a screening set cannot be interpreted — part of the apparent accuracy may simply be recall.
5. **A documented error floor.** We state explicitly what accuracy is achievable and why, and we report both the multiplicative error and the range-normalised error rather than choosing whichever is more flattering.

What is deliberately the same: the CGCNN architecture and hyperparameters, the training data, the $\log_{10}$ target, and the 70/15/15 split ratio. The point of a reproduction is that differences should be deliberate and few.

8 Reproducing this work

```
conda env create -f environment.yml && conda activate pink-cgcnn
./run_pipeline.sh --predict      # ~2 min, uses committed checkpoints
./run_pipeline.sh               # retrain: ~45 min GPU, ~9 h CPU
```

The six trained checkpoints (~ 380 KB each) are committed, so every figure and number above regenerates in about two minutes without training. The 240 MB graph cache is not committed; it rebuilds deterministically in ~ 6 minutes.

8.1 A note on compute

This work was developed on a 2016 dual-core laptop CPU, where one model takes ~ 2 hours. The same model trains in ~ 7 minutes on a free Colab T4 — all six in 42 minutes. `colab/PINK_CGCNN.ipynb` runs the pipeline there and returns the checkpoints; it is *generated* from the source files rather than maintained by hand, so it cannot drift out of sync with the featuriser it is meant to reproduce.

Two portability details worth recording. Python block-buffers stdout at 8 KB when it is not a terminal, and a 200-epoch run prints only ~ 2 KB — so without `python -u` a healthy remote run is indistinguishable from a hung one. And `torch.load` changed its `weights_only` default in torch 2.6, so every load of our own artifacts sets it explicitly; otherwise code that works locally fails on the newer torch of a hosted GPU runtime.

9 Limitations and next steps

- **Soft materials remain the weakest region.** Residuals fan out below ~ 30 GPa. This is partly genuine label noise — DFT is least reliable for soft, anharmonic crystals — but it is also precisely the ultralow- κ regime PINK targets, so it is the most valuable place to improve.
- **Only $\sim 11k$ labelled crystals exist.** Accuracy is now limited by data and label quality rather than by the model. Semi-supervised pre-training on unlabelled structures is the obvious lever.
- **Stage 2 is not yet reimplemented.** The Slack-model half still runs on the authors' code. Substituting our `pink_moduli_predictions.csv` into it, and propagating our per-crystal uncertainties through to κ_L , is the natural next step — and something the original pipeline cannot do.

Reference. Liu, Y.; Wang, X.; Hao, Y.; Li, X.; Sun, J.; Lookman, T.; Ding, X.; Gao, Z. *PINK: physics-informed machine learning for lattice thermal conductivity*. J. Mater. Inf. 2025, 5, 12. [doi:10.20517/jmi.2024.86](https://doi.org/10.20517/jmi.2024.86)

Underlying data. Matbench elastic benchmarks `matbench_log_kvrh` and `matbench_log_gvrh` (Dunn *et al.*, npj Comput. Mater. 2020), derived from the Materials Project.